

Controllers

Authcontrollers.js

```
const User = require('../models/User');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const generateToken = (id, isAdmin) => {
  return jwt.sign({ id, isAdmin }, process.env.JWT_SECRET, { expiresIn: '30d' });
};

exports.registerUser = async (req, res) => {
  const { name, email, password, isAdmin } = req.body;
  try {
    const userExists = await User.findOne({ email });
    if (userExists) return res.status(400).json({ message: 'User already exists' });

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    const user = await User.create({ name, email, password: hashedPassword, isAdmin });

    res.status(201).json({ _id: user._id, name: user.name, email: user.email,
    isAdmin: user.isAdmin, token: generateToken(user._id, user.isAdmin) });
  } catch (error) { res.status(500).json({ message: error.message }); }
};

exports.loginUser = async (req, res) => {
  const { email, password } = req.body;
  try {
    const user = await User.findOne({ email });
    if (user && (await bcrypt.compare(password, user.password))) {
      res.json({ _id: user._id, name: user.name, email: user.email, isAdmin:
      user.isAdmin, token: generateToken(user._id, user.isAdmin) });
    } else {
      res.status(401).json({ message: 'Invalid email or password' });
    }
  } catch (error) { res.status(500).json({ message: error.message }); }
};
```

BookingController.js

```
const Booking = require('../models/Booking');
const Show = require('../models/Show');

exports.createBooking = async (req, res) => {
  const { showId, selectedSeats } = req.body;
  try {
    const show = await Show.findById(showId);
    if (!show) return res.status(404).json({ message: 'Show not found' });

    const alreadyBooked = show.bookedSeats.some(seat =>
selectedSeats.includes(seat));
    if (alreadyBooked) {
      return res.status(400).json({ message: 'One or more selected seats are
already booked' });
    }

    show.bookedSeats.push(...selectedSeats);
    await show.save();

    const totalPrice = show.price * selectedSeats.length;

    const booking = await Booking.create({
      user: req.user.id,
      show: showId,
      seats: selectedSeats,
      totalPrice
    });

    res.status(201).json(booking);
  } catch (error) { res.status(500).json({ message: error.message }); }
};

exports.getMyBookings = async (req, res) => {
  try {
    const bookings = await Booking.find({ user: req.user.id }).populate({ path:
'show', populate: { path: 'movie' } });
    res.json(bookings);
  } catch (error) { res.status(500).json({ message: error.message }); }
};

exports.getAllBookings = async (req, res) => {
```

```

    try {
      const bookings = await Booking.find({}).populate('user', 'name email').populate({ path: 'show', populate: { path: 'movie' } });
      res.json(bookings);
    } catch (error) { res.status(500).json({ message: error.message }); }
  });

exports.removeSeatFromBooking = async (req, res) => {
  try {
    const { bookingId, seatId } = req.params;

    const booking = await Booking.findById(bookingId).populate('show');
    if (!booking) return res.status(404).json({ message: 'Booking not found' });

    const show = booking.show;
    if (!show) return res.status(404).json({ message: 'Associated show not found' });

    if (!booking.seats.includes(seatId)) {
      return res.status(400).json({ message: 'Seat not found in this booking' });
    }

    // Remove seat from booking
    booking.seats = booking.seats.filter(seat => seat !== seatId);

    // Lower the total price
    booking.totalPrice -= show.price;

    // Make the seat available in the show
    show.bookedSeats = show.bookedSeats.filter(seat => seat !== seatId);
    await show.save();

    if (booking.seats.length === 0) {
      await Booking.findByIdAndDelete(bookingId);
      return res.json({ message: `Seat ${seatId} successfully removed. Booking was completely deleted as no seats remain. ₹${show.price} refunded.` });
    } else {
      await booking.save();
      return res.json({ message: `Seat ${seatId} successfully removed. ₹${show.price} refunded.` });
    }
  }
};

```

```
    } catch (error) {
      res.status(500).json({ message: error.message });
    }
  };
};
```

MovieController.js

```
const Movie = require('../models/Movie');

exports.getMovies = async (req, res) => {
  try {
    const movies = await Movie.find({});
    res.json(movies);
  } catch (error) { res.status(500).json({ message: error.message }); }
};

exports.getMovieById = async (req, res) => {
  try {
    const movie = await Movie.findById(req.params.id);
    if (movie) res.json(movie);
    else res.status(404).json({ message: 'Movie not found' });
  } catch (error) { res.status(500).json({ message: error.message }); }
};

exports.createMovie = async (req, res) => {
  try {
    const movie = await Movie.create(req.body);
    res.status(201).json(movie);
  } catch (error) { res.status(500).json({ message: error.message }); }
};

exports.updateMovie = async (req, res) => {
  try {
    const movie = await Movie.findByIdAndUpdate(req.params.id, req.body, { new: true });
    res.json(movie);
  } catch (error) { res.status(500).json({ message: error.message }); }
};

exports.deleteMovie = async (req, res) => {
  try {
```

```
    await Movie.findByIdAndDelete(req.params.id);
    res.json({ message: 'Movie removed' });
  } catch (error) { res.status(500).json({ message: error.message }); }
};
```

Showcontroller.js

```
const Show = require('../models/Show');

exports.getShowsByMovie = async (req, res) => {
  try {
    const shows = await Show.find({ movie: req.params.movieId }).populate('movie');
    res.json(shows);
  } catch (error) { res.status(500).json({ message: error.message }); }
};

exports.createShow = async (req, res) => {
  try {
    const show = await Show.create(req.body);
    res.status(201).json(show);
  } catch (error) { res.status(500).json({ message: error.message }); }
};
```